UNIVERSIDAD DE LA HABANA

Facultad de Matemática y Computación

BASES DE DATOS II

Informe: Diseño de la base de datos

Gestión para la Generación Automática de Rutas de Reparto

Curso: 2026–2027
Equipo: Digimons

Integrante	Grupo	Telegram	GitHub
Karla Díaz Saura	C312	@kdiazsr	Krli01
Manuel Enrique Cardoso Martínez	C312	@ManuEC04	ManuEC04
Leonardo Peláez Ascención	C312	@leosoyyoxd	Brand-Leo
Jorge Luis Herrera Cecilia	C312	@Jorge_Lhc	Jorge-Luis-Herrera
José Alejandro Albanés Febles	C311	@cuban_developer	JoseAlejandro0508

Informe elaborado conforme a las Orientaciones Metodológicas de Bases de Datos II.

Índice

1 Título del proyecto **2**

2 Análisis y reformulación de requerimientos **2**

2.1 Descripción general 2

2.2 Actores y responsabilidades 2

2.3 Requerimientos funcionales reformulados 2

2.4 Requerimientos informacionales 3

2.5 Requerimientos no funcionales 3

3 Stack tecnológico y arquitectura propuesta **4**

3.1 Stack tecnológico 4

3.2 Arquitectura y comunicación 5

3.3 Patrones, calidad y operación 5

4 Modelo Conceptual Relacional Extendido (MERX) **5**

4.1 Vista conceptual 5

4.2 Entidades y relaciones principales 6

5 Especificación de restricciones de integridad **7**

5.1 Integridad de entidad 7

5.2 Integridad referencial 7

5.3 Restricciones de negocio 7

6 Valoración de la corrección del diseño **8**

6.1 Dependencias funcionales y normalización 8

6.2 Preservación y reunión sin pérdida 8

6.3 Decisiones de implementación recomendadas 8

7 Conclusiones **9**

Fuentes de trabajo utilizadas **9**

1 Título del proyecto

Gestión para la Generación Automática de Rutas de Reparto.

2 Análisis y reformulación de requerimientos

2.1 Descripción general

El proyecto nace de una necesidad muy concreta: organizar mejor una operación de reparto en la que intervienen pedidos, centros, zonas, coordinadores, supervisores y repartidores. La aplicación web permitirá registrar pedidos y paradas, organizar la cobertura territorial, generar rutas, validar la planificación, ejecutar las entregas y analizar sus resultados. La base de datos acompañará todo ese recorrido, de modo que cada decisión pueda rastrearse desde el ingreso del pedido hasta la entrega o su reprogramación.

La solución combina un modelo relacional para la información transaccional con un motor de grafos externo para la red dinámica de vías y puntos de tránsito. Esta separación permite consultar y recorrer eficientemente rutas alternativas sin introducir en el esquema relacional una estructura que cambia y se navega constantemente.

2.2 Actores y responsabilidades

Actor	Responsabilidades principales
Administrador	Gestionar usuarios y parámetros generales; impedir pedidos duplicados y asignaciones incompatibles de rutas.
Coordinador logístico	Registrar y clasificar pedidos, operar en centros autorizados, seleccionar cargas, generar rutas y resolver reprogramaciones asignadas.
Supervisor de operaciones	Revisar rutas, aprobarlas, rechazarlas o solicitar modificaciones, dejando constancia de sus observaciones.
Repartidor	Consultar su ruta, ejecutar las paradas, registrar entregas e incidencias y solicitar reprogramaciones.

2.3 Requerimientos funcionales reformulados

1. **Autenticación y autorización.** El sistema centralizará la identidad de los usuarios y aplicará permisos según cuatro roles: administrador, coordinador logístico, supervisor de operaciones y repartidor. La jerarquía será total y disjunta: todo usuario operativo pertenece a un subtipo y no pertenece simultáneamente a dos subtipos incompatibles.
2. **Gestión de centros, zonas y paradas.** Se registrarán centros de distribución, sus zonas cubiertas y las paradas de destino. Un centro puede cubrir varias zonas y una zona puede ser atendida por varios centros.

3. **Registro de pedidos.** Un coordinador autorizado ingresará cada pedido, indicando descripción, tipo de entrega (paquetería, documentos o perecederos), prioridad (baja, media o urgente), centro de origen y parada de destino.
4. **Generación automática de rutas.** El sistema seleccionará pedidos y generará una ruta considerando capacidad del vehículo, distribución territorial, tipos de entrega, prioridad y criterios de equilibrio. La parametrización utilizada quedará almacenada junto a la ruta.
5. **Supervisión.** Toda ruta propuesta podrá ser revisada por un supervisor. El resultado de la revisión, la fecha y las observaciones se conservarán para auditoría.
6. **Asignación y ejecución.** Una ruta aprobada podrá asignarse a un repartidor habilitado para el centro correspondiente. El sistema impedirá que tenga dos rutas activas incompatibles.
7. **Entregas y reprogramaciones.** El repartidor registrará el resultado de cada entrega y sus observaciones. Si falla, podrá crear una solicitud de reprogramación y designar el coordinador que la resolverá; se conservará el resultado manual.
8. **Reportes y exportación.** Todos los usuarios podrán consultar resultados tabulares y gráficos, ordenar columnas y exportar la información a PDF.

2.4 Requerimientos informacionales

La base de datos deberá soportar, como mínimo, las siguientes consultas:

1. Rutas generadas para un centro, con creador, fecha de creación y parámetros.
2. Pedidos más gestionados en rutas finales, clasificados por prioridad y zona.
3. Rutas validadas por un supervisor, con fecha y observaciones.
4. Desempeño de repartidores en una ruta, agrupado por prioridad y tasa de entrega a tiempo.
5. Comparación de rutas de varios centros por zona, prioridad y cumplimiento de criterios.
6. Correlación entre prioridad y rendimiento por centro; identificación de los diez pedidos con mayor incumplimiento, su coordinador y centro; y comparación entre repartidores que solicitaron reprogramación y el promedio de entrega a tiempo de sus flotas.

2.5 Requerimientos no funcionales

- **Rendimiento y concurrencia:** las consultas frecuentes sobre disponibilidad y entregas deben responder con baja latencia aun con varios coordinadores conectados.
- **Disponibilidad:** el estado de repartidores y paradas en curso debe reflejarse oportunamente durante la jornada.
- **Escalabilidad y mantenibilidad:** la arquitectura debe estar desacoplada, documentada, modular y preparada para incorporar nuevas funcionalidades.

- **Multiplataforma:** la aplicación web debe poder utilizarse desde los entornos previstos para el equipo y los usuarios.
- **Calidad del desarrollo:** se exige control de versiones, planificación, documentación del código, al menos dos patrones de diseño y pruebas unitarias de backend y frontend.
- **Red territorial:** la red de conexiones se gestionará independientemente mediante una estructura de grafos adecuada a recorridos y búsquedas dinámicas.

3 Stack tecnológico y arquitectura propuesta

3.1 Stack tecnológico

La solución se implementará como una aplicación web multiplataforma organizada en servicios desacoplados. La selección tecnológica responde a la necesidad de construir una interfaz mantenible, una API con validación estricta, un modelo relacional transaccional y un componente especializado para la red dinámica de vías.

Capa	Tecnologías	Responsabilidad
Presentación	React, TypeScript, Vite, React Router	Interfaz web, navegación, formularios, consulta de rutas y visualización de reportes.
Estado y datos	TanStack Query	Consumo de la API, caché, sincronización y estados de carga o error.
Visualización	Recharts	Gráficos de prioridades, cumplimiento, desempeño y comparación de centros.
Backend	Python, FastAPI, Pydantic	API REST, validación de datos, autenticación y coordinación de los casos de uso.
Persistencia relacional	PostgreSQL, SQLAlchemy, Alembic	Usuarios, pedidos, centros, rutas, entregas, reprogramaciones y restricciones de integridad.
Persistencia de grafos	Neo4j	Zonas, vías, puntos de tránsito, distancias, tiempos estimados y trayectos alternativos.
Pruebas	Pytest, Vitest, React Testing Library	Pruebas unitarias del backend y frontend, además de validación de reglas críticas.
Ejecución	Docker Compose	Entorno reproducible para frontend, backend, PostgreSQL y Neo4j.
Colaboración	GitHub, GitHub Projects, GitHub Actions	Control de versiones, planificación, revisión y ejecución automatizada de pruebas.

3.2 Arquitectura y comunicación

La arquitectura seguirá un enfoque por capas. React se comunicará con FastAPI mediante una API REST sobre HTTPS. FastAPI separará las rutas HTTP, los servicios de aplicación, los repositorios y las integraciones con PostgreSQL y Neo4j. PostgreSQL será la fuente de verdad para las operaciones transaccionales; Neo4j se utilizará para recorrer la red territorial y sugerir trayectos alternativos. La comunicación entre ambos almacenes se coordinará desde la capa de servicios y no desde la interfaz de usuario.

- **Capa de presentación:** componentes React tipados con TypeScript, formularios con validación y vistas de operación y reportes.
- **Capa de API:** endpoints REST versionados, esquemas Pydantic y control uniforme de errores.
- **Capa de aplicación:** casos de uso para registrar pedidos, generar rutas, validar rutas, registrar entregas y resolver reprogramaciones.
- **Capa de persistencia:** repositorios separados para PostgreSQL y Neo4j, con transacciones relacionales y consultas de grafos especializadas.
- **Capa de seguridad:** autenticación mediante tokens de acceso y renovación, contraseñas protegidas con Argon2 y autorización según rol y centro habilitado.

3.3 Patrones, calidad y operación

Se aplicarán el patrón **Repository** para aislar el acceso a datos, **Service Layer** para concentrar las reglas de negocio y **Strategy** para permitir nuevos criterios de equilibrio sin modificar el flujo general de generación de rutas. El código se organizará por responsabilidades, con nombres descriptivos, documentación de funciones públicas y validaciones en los límites de cada servicio.

Las pruebas unitarias cubrirán como mínimo la autorización por rol, la prevención de pedidos duplicados, la asignación exclusiva de repartidores, la generación según criterios, el registro de entregas y la creación de reprogramaciones. El frontend se probará con Vitest y React Testing Library; el backend, con Pytest. GitHub Actions ejecutará estas pruebas automáticamente ante cambios relevantes.

La configuración se gestionará mediante variables de entorno y un archivo de ejemplo. Docker Compose permitirá levantar los servicios de forma reproducible. El repositorio deberá incluir el fichero de dependencias, el archivo ‘.gitignore‘, los servicios necesarios para el entorno local y una planificación visible en GitHub Projects con tareas, responsables, estados, fechas y milestones.

4 Modelo Conceptual Relacional Extendido (MERX)

4.1 Vista conceptual

El MERX se organiza alrededor de la entidad raíz **USUARIO** y una especialización ISA total y disjunta en COORDINADOR_LOGISTICO, REPARTIDOR, SUPERVISOR_OPERACIONES

y ADMINISTRADOR. Las entidades territoriales y operativas se conectan mediante relaciones con cardinalidades explícitas. Los atributos propios de las relaciones N:M se representan como atributos de **RUTA_PARADA** y **RUTA_CRITERIO** en el esquema relacional.

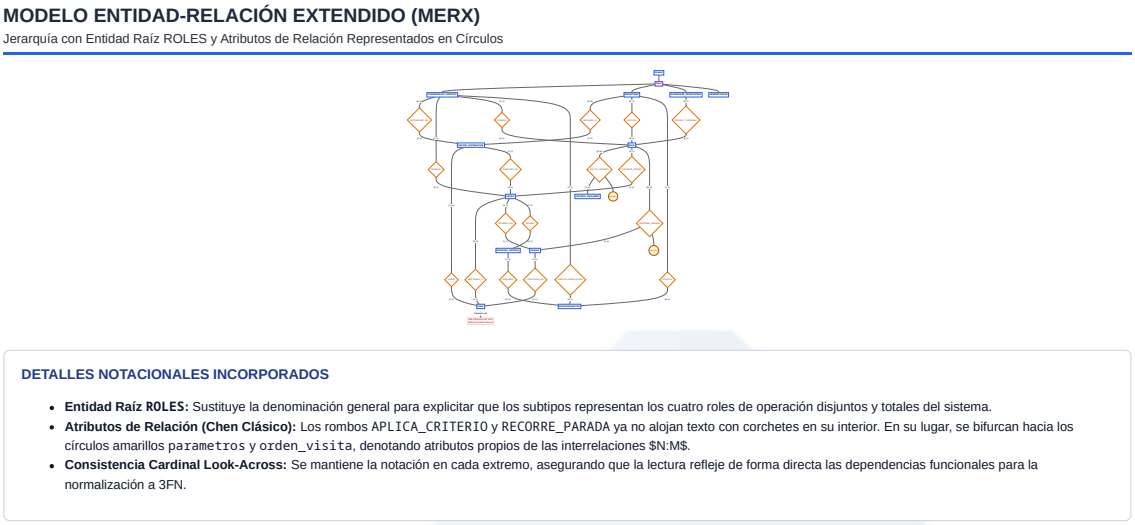


Figura 1: Modelo Entidad-Relación Extendido (MERX).

4.2 Entidades y relaciones principales

- **Usuarios y roles:** USUARIO centraliza nombre, correo, credenciales, estado y renovación de sesión. Cada subtipo agrega sus atributos específicos.
- **Territorio:** CENTRO_DISTRIBUCION se relaciona con ZONA mediante CUBRE; PARADA se localiza en una zona. La topología vial queda fuera del modelo relacional.
- **Carga:** PEDIDO es ingresado por un coordinador, radicado en un centro y destinado a una parada. Una ruta contiene cero o más pedidos seleccionados, resolviendo la trampa de conexión entre planificación y ejecución.
- **Planificación:** RUTA es generada por un coordinador, se origina en un centro, puede ser validada por un supervisor y ejecutada por un repartidor. RUTA_PARADA conserva el orden de visita y RUTA_CRITERIO conserva la parametrización aplicada.

- **Trazabilidad:** cada REGISTRO_ENTREGA documenta un pedido; una entrega fallida puede originar una REPROGRAMACION solicitada por el repartidor y resuelta por un coordinador.

5 Especificación de restricciones de integridad

5.1 Integridad de entidad

Toda tabla posee una clave primaria no nula y única. Las claves principales son ‘id_usuario‘, ‘id_coordinador‘, ‘id_repartidor‘, ‘id_supervisor‘, ‘id_admin‘, ‘id_centro‘, ‘id_zona‘, ‘id_parada‘, ‘id_pedido‘, ‘id_criterio‘, ‘id_ruta‘, ‘id_entrega‘ e ‘id_reprogramacion‘. Las tablas asociativas utilizan claves compuestas: ‘(id_centro,id_zona)‘, ‘(id_ruta,id_parada)‘, ‘(id_ruta,id_criterio)‘, entre otras.

5.2 Integridad referencial

- Cada subtipo de usuario referencia exactamente un USUARIO existente.
- PEDIDO referencia un coordinador, centro y parada existentes; la ruta es nullable mientras el pedido no haya sido despachado.
- PARADA referencia una ZONA existente. CENTRO_ZONA relaciona únicamente centros y zonas existentes.
- RUTA referencia al coordinador que la genera y al centro de origen; supervisor y repartidor pueden ser nulos hasta completar esas etapas.
- RUTA_PARADA y RUTA_CRITERIO solo pueden referenciar rutas, paradas y criterios existentes.
- REGISTRO_ENTREGA referencia un PEDIDO existente y REPROGRAMACION referencia una entrega existente.

5.3 Restricciones de negocio

1. **No duplicados:** se definirá una restricción única sobre los atributos que identifiquen el mismo pedido dentro del negocio, además de validación de aplicación.
2. **Autorización:** un coordinador solo puede registrar pedidos o generar rutas para centros presentes en COORDINADOR_CENTRO.
3. **Asignación exclusiva:** un repartidor no puede tener más de una ruta activa incompatible; se garantizará mediante validación transaccional y una restricción de unicidad parcial si el gestor lo permite.
4. **Consistencia geográfica:** la parada debe pertenecer a una zona válida y la combinación centro-zona debe existir cuando el pedido o ruta se asignen al centro.

5. **Persistencia de criterios:** los parámetros se almacenan como una instantánea en RUTA_CRITERIO, de modo que cambios posteriores en el criterio no alteren la trazabilidad histórica.
6. **Reprogramación válida:** solo una entrega registrada y no completada puede generar una solicitud; el estado y la fecha de resolución deben ser coherentes.
7. **Estados controlados:** prioridad, tipo de entrega, estado de pedido, estado de ruta y resultado de entrega deben restringirse a dominios definidos por el sistema.

6 Valoración de la corrección del diseño

6.1 Dependencias funcionales y normalización

A partir del universo de atributos se obtiene el siguiente conjunto representativo de dependencias funcionales:

$$\begin{aligned}
\texttt{id_usuario} &\rightarrow \text{datos generales del usuario},\\
\texttt{id_pedido} &\rightarrow \text{datos del pedido, coordinador, centro y parada},\\
\texttt{id_parada} &\rightarrow \texttt{id_zona},\\
\texttt{id_ruta} &\rightarrow \text{fechas, capacidad, estado, coordinador y centro},\\
(\texttt{id_ruta}, \texttt{id_parada}) &\rightarrow \texttt{orden_visita},\\
(\texttt{id_ruta}, \texttt{id_criterio}) &\rightarrow \texttt{parametros_configuracion}.
\end{aligned}$$

La dependencia ‘id_pedido →id_zona‘ no se conserva como dependencia independiente porque se deriva transitivamente de ‘id_pedido →id_parada‘ y ‘id_parada →id_zona‘. El esquema resultante elimina esa redundancia y evita anomalías de actualización.

La síntesis por determinantes produce, entre otras, las tablas USUARIO, los subtipos de rol, CENTRO_DISTRIBUCION, ZONA, CENTRO_ZONA, PARADA, PEDIDO, CRITERIO_EQUILIBRIO, RUTA, RUTA_PARADA, RUTA_CRITERIO, COORDINADOR_CENTR REPARTIDOR_CENTRO, REGISTRO_ENTREGA y REPROGRAMACION. Cada relación contiene atributos que dependen de su clave, de toda su clave y de ningún atributo no clave; por tanto, satisface 3FN bajo las dependencias declaradas.

6.2 Preservación y reunión sin pérdida

Las dependencias se preservan porque cada determinante y sus atributos dependientes aparecen en una relación del esquema sintetizado. Las claves foráneas conservan los vínculos semánticos y permiten reconstruir la información mediante reuniones sin pérdida. La asociación directa RUTA–PEDIDO evita perder la relación entre una expedición y su carga cuando una parada concentra pedidos de distintos recorridos.

6.3 Decisiones de implementación recomendadas

- Usar transacciones para generar rutas, asignar repartidores y registrar entregas.

- Indexar claves foráneas, estados operativos, fechas, centro y prioridad para soportar los reportes.
- Aplicar control de concurrencia al consultar disponibilidad y al confirmar asignaciones.
- Mantener la red vial en el motor de grafos y almacenar en el modelo relacional las referencias necesarias para enlazar paradas y zonas.
- Implementar una arquitectura desacoplada por capas o módulos, aplicar al menos dos patrones y acompañar backend y frontend con pruebas unitarias.

7 Conclusiones

Este diseño busca que la base de datos refleje la operación real y no solo una lista de tablas. Los roles representan responsabilidades diferentes; los pedidos conservan su origen, destino y estado; las rutas guardan los criterios con los que fueron generadas; y las entregas permiten saber qué ocurrió en cada parada. Así, la información no se pierde cuando una ruta cambia, una entrega falla o debe solicitarse una reprogramación.

La especialización ISA evita repetir los datos comunes de los usuarios, mientras que las relaciones asociativas permiten representar correctamente las rutas, sus paradas y los criterios aplicados. La red vial se mantiene en un motor de grafos porque sus conexiones y recorridos cambian con mayor frecuencia que los datos administrativos. Esta separación mantiene el modelo relacional claro y deja abierta la posibilidad de buscar alternativas cuando una entrega no puede completarse como estaba prevista.

Las claves, referencias y restricciones de negocio protegen la consistencia de la información; la normalización a 3FN reduce duplicidades y evita anomalías de actualización. Además, el stack tecnológico propuesto permite llevar el modelo a una aplicación mantenible, con una interfaz accesible, una API validada, pruebas automatizadas y un entorno reproducible. El resultado no pretende cerrar todas las decisiones de implementación, pero sí ofrece una base sólida, comprensible y verificable para construir el sistema.

Fuentes de trabajo utilizadas

- International Council on Systems Engineering. *Requirements Engineering*, SEBoK v2.14. Disponible en: https://www.sebokwiki.org/wiki/Requirements_Engineering. Consultado el 9 de octubre de 2026.
- University of Cape Town. *Chapter 6: Entity-Relationship Modelling*. Material académico de bases de datos. Disponible en: https://www.cs.uct.ac.za/mit_notes/database/htmls/chp06.html. Consultado el 9 de octubre de 2026.
- University of Cape Town. *Chapter 7: Enhanced Entity-Relationship Modelling*. Material académico sobre especialización, generalización y restricciones ISA. Disponible en: https://www.cs.uct.ac.za/mit_notes/database/htmls/chp07.html. Consultado el 9 de octubre de 2026.
- University of Cape Town. *Chapter 8: Data Normalisation*. Material académico sobre dependencias funcionales y 1FN–3FN. Disponible en: https://www.cs.uct.ac.za/

mit_notes/database/htmls/chp08.html. Consultado el 9 de octubre de 2026.

- PostgreSQL Global Development Group. *Data Definition and Constraints*. Documentación oficial sobre claves, restricciones e integridad referencial. Disponible en: https://www.postgresql.org/docs/current/ddl-constraints.html. Consultado el 9 de octubre de 2026.
- Neo4j, Inc. *Get Started with Neo4j*. Documentación oficial sobre bases de datos de grafos y Cypher. Disponible en: https://neo4j.com/docs/getting-started/current/. Consultado el 9 de octubre de 2026.
- React. *React Documentation: Learn*. Documentación oficial sobre componentes y estado de interfaces. Disponible en: https://react.dev/learn. Consultado el 9 de octubre de 2026.
- FastAPI. *FastAPI Documentation*. Documentación oficial sobre APIs, validación y OpenAPI. Disponible en: https://fastapi.tiangolo.com/. Consultado el 9 de octubre de 2026.
- Docker. *Get Started*. Documentación oficial sobre contenedores y entornos reproducibles. Disponible en: https://docs.docker.com/get-started/. Consultado el 9 de octubre de 2026.
- GitHub. *About Projects*. Documentación oficial sobre planificación mediante tableros, milestones y seguimiento de tareas. Disponible en: https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects. Consultado el 9 de octubre de 2026.

MATCOM